

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems **NAME:**K.SARANYA
COURSE CODE: CSA0501 **REG NO:**192424409

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Mock Interview (Low)
Assessment Tool Weightage: 30%

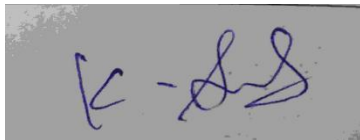
Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'	BL3 – Apply	5
2	Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'	BL4 – Analyze	5
3	Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.'	BL3 – Apply	5
4	Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'	BL3 – Apply	5
5	Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?'	BL4 – Analyze	5
6	Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'	BL4 – Analyze	5
7	Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'	BL4 – Analyze	5

8	Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'	BL3 – Apply	5
9	Interviewer: 'What is a serializability schedule? How do you test for conflict serializability using a precedence graph?'	BL4 – Analyze	5
10	Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?'	BL4 – Analyze	5

Code of Conduct:

*I **K.SARANYA(192424409)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*



Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

Q1. Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?

Overview

Query processing is the sequence of activities by which a DBMS translates a high-level query (typically SQL) into a low-level, executable set of operations on the physical database, and then executes it to produce a result. The goal is correctness first, and efficiency second — many equivalent execution strategies exist for the same query, and the DBMS must choose one that minimises resource usage (I/O, CPU, memory).

Steps of Query Processing

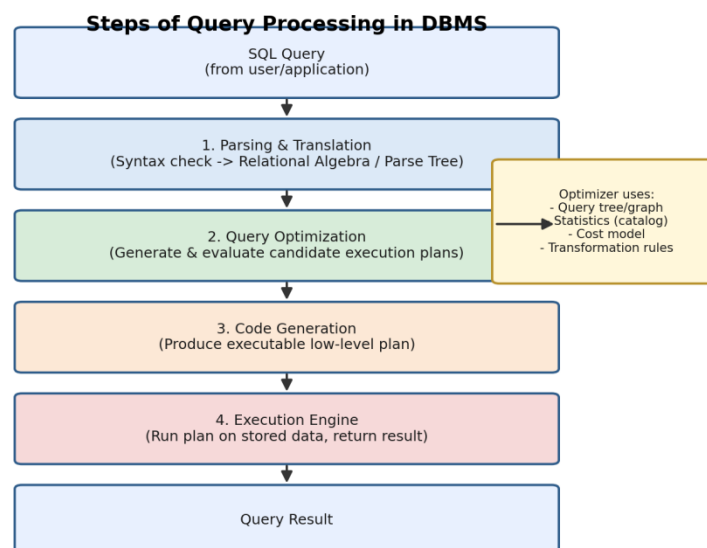


Figure 1.1: Stages of Query Processing in a DBMS

- **Parsing and Translation:** The SQL query is checked for syntactic and semantic correctness (do the referenced tables/columns exist, are types compatible). The parser converts it into an internal representation such as a parse tree, which is then translated into an equivalent expression in relational algebra or a query graph.
- **Query Optimization:** The relational algebra expression can be implemented in many different ways (different join orders, different access paths such as index scan vs full scan). The optimizer enumerates candidate execution plans and estimates the cost of each, then selects the plan with the lowest estimated cost.
- **Code Generation:** The chosen execution plan is converted into actual executable code or an internal query-execution plan (a tree of physical operators such as Index-Scan, Hash-Join, Sort) that the execution engine can run directly.

- Execution: The execution engine runs the generated code against the actual stored data (tables, indexes) and produces the final result set, which is returned to the user or application.

How the Query Optimizer Decides the Best Plan

The optimizer's job is to search the space of semantically equivalent execution plans and pick the cheapest one. It typically works in the following way:

- Plan Space Generation: Using equivalence rules of relational algebra (e.g., selection can be pushed below a join, joins are associative and commutative), the optimizer generates multiple logically equivalent query plans from the original expression.
- Statistics Collection: The system catalog stores statistics about each relation — number of tuples, number of distinct values per attribute, index availability, and data distribution histograms.
- Cost Estimation: For every candidate plan, the optimizer estimates the cost (usually dominated by disk I/O, plus CPU and memory) using formulas based on the statistics — e.g., cost of a nested-loop join vs a hash join vs a sort-merge join for the given relation sizes.
- Plan Selection: Techniques such as dynamic programming (used for join-order selection to avoid exponential blow-up) are used to prune the search space and select the plan with minimum estimated cost.
- Execution: The selected physical plan is handed to the code generator/execution engine.

In short, the optimizer decides the best plan by balancing correctness (all candidate plans must be equivalent to the original query) against efficiency (choosing the plan predicted to consume the least time/resources based on cost estimation over collected statistics).

Q2. What is the difference between heuristic optimization and cost-based optimization? When would you use each?

Heuristic (Rule-Based) Optimization

Heuristic optimization applies a fixed set of transformation rules to the query tree, based on general principles that are almost always beneficial, without computing an actual cost.

Common heuristics include performing selection and projection operations as early as possible (reducing the size of intermediate relations), avoiding Cartesian products in favour of joins, and reordering operations to reduce the size of data flowing through the pipeline.

Cost-Based Optimization

Cost-based optimization goes further: it generates multiple candidate execution plans (e.g., different join orders, different access methods) and estimates the actual cost of each plan

using statistics from the system catalog (relation cardinality, index selectivity, available memory). It then picks the plan with the lowest estimated cost.

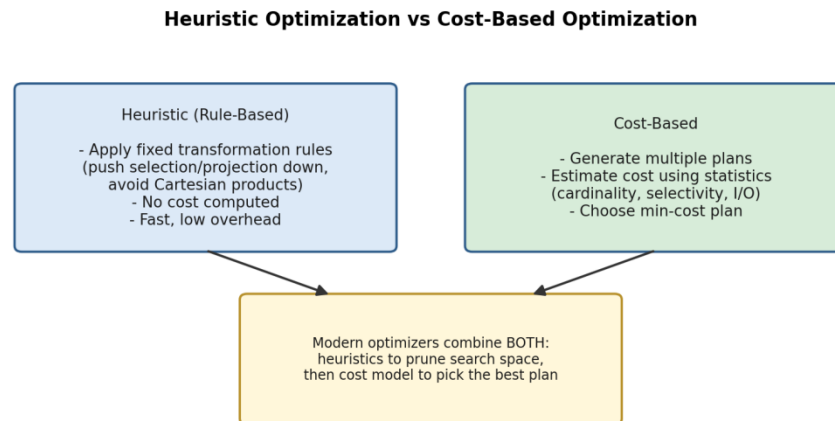


Figure 2.1: Heuristic vs Cost-Based Optimization

Comparison Table

Aspect	Heuristic Optimization	Cost-Based Optimization
Basis	Fixed transformation rules	Estimated execution cost using statistics
Statistics needed	Not required	Required (cardinality, selectivity, indexes)
Overhead	Low - fast to apply	Higher - must evaluate multiple plans
Accuracy	May not always give optimal plan	Generally produces near-optimal plan
Best suited for	Simple queries, systems with no statistics	Complex queries with joins on large tables

When to Use Each

- Use heuristic optimization when the system has limited statistics, for simple queries, or as a first-pass pruning step to reduce the number of candidate plans before applying cost estimation (this is exactly how most commercial optimizers work).
- Use cost-based optimization when dealing with complex, multi-way joins over large tables where the difference between a good and a bad plan can be orders of magnitude in execution time, and where reliable statistics are available in the catalog.

In practice, modern DBMS optimizers (Oracle, PostgreSQL, MySQL) use a hybrid approach: heuristics first prune obviously bad plans, and cost-based estimation is then used to choose among the remaining candidates.

Q3. How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.

Implementing Database Connectivity in a Real Project

In a real-world application, database connectivity is implemented through a standard API layer that lets application code talk to the DBMS without knowing its internal implementation. A typical implementation approach:

- Choose a connectivity standard appropriate to the language/platform: JDBC for Java applications, ODBC for C/C++/.NET or cross-language environments, or an ORM (Hibernate, Entity Framework) built on top of these.
- Load and register the appropriate driver for the target database (e.g., MySQL Connector/J for JDBC, or a vendor-supplied ODBC driver).
- Establish a connection using a connection string that specifies host, port, database name, and credentials — typically managed through a connection pool for performance (e.g., HikariCP for Java applications).
- Execute SQL statements via prepared statements (to prevent SQL injection and allow query plan reuse), process the ResultSet/recordset returned, and handle transactions explicitly (commit/rollback) where multiple statements must succeed or fail together.
- Close connections/release them back to the pool, and handle exceptions (connection timeouts, constraint violations) gracefully with retries or user-facing error messages.

JDBC vs ODBC

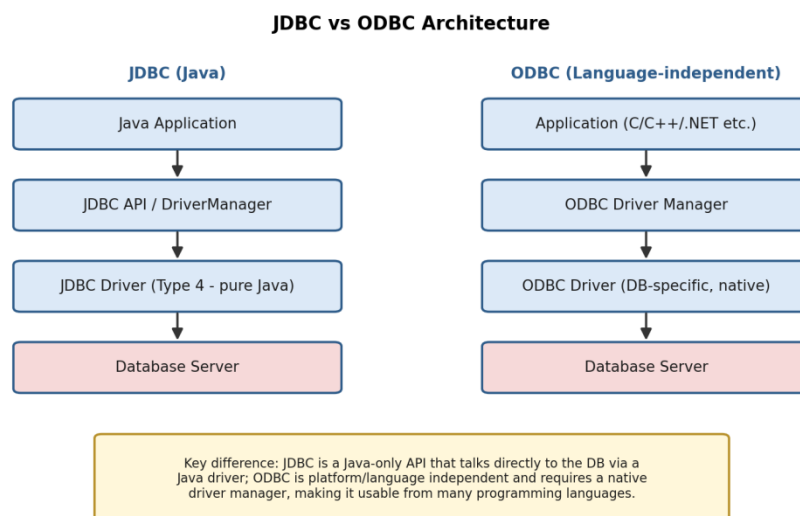


Figure 3.1: JDBC and ODBC Connectivity Architecture

Aspect	JDBC	ODBC
Full form	Java Database Connectivity	Open Database Connectivity
Language dependency	Java-specific API	Language-independent (C-based API)
Driver type	Pure Java driver (Type 4 is common)	Native driver, needs driver manager
Platform	Platform-independent (JVM)	Often platform-dependent (native libraries)
Ease of use	Simpler for Java developers, object-oriented	Requires more setup, lower-level C API
Performance	Good, some JVM overhead	Very fast - close to native calls
Typical use case	Java enterprise applications	Applications in multiple languages (VB, C++, Python via pyodbc)

In summary, JDBC is the natural choice inside a Java-based project because it is portable across operating systems and integrates cleanly with the Java ecosystem, while ODBC is the right choice when the application is written in a non-Java language or when connectivity must be shared across multiple languages/tools on the same machine.

Q4. What are ACID properties? Give a real-world example where each property is critical.

ACID Properties

ACID is an acronym describing the four properties that guarantee reliable processing of database transactions, ensuring that the database remains correct even in the presence of concurrent access, errors, and system failures.

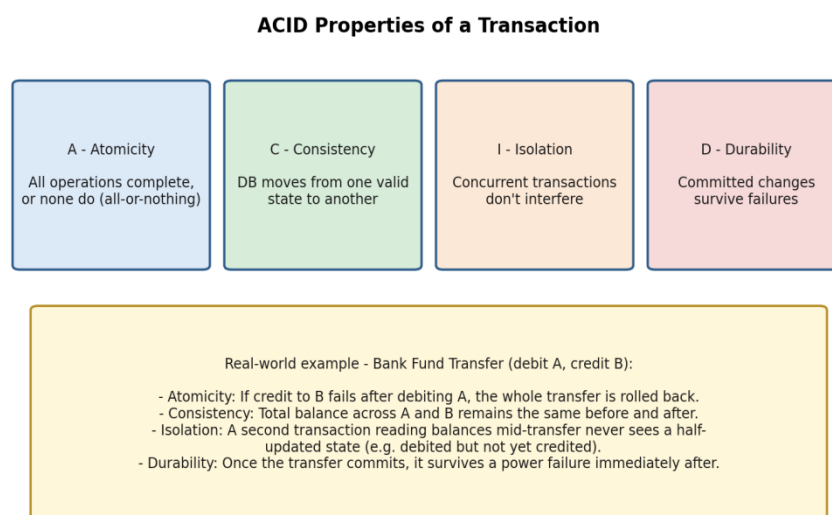


Figure 4.1: ACID Properties and a Fund-Transfer Example

Detailed Explanation with Real-World Examples

Property	Meaning	Real-World Example
Atomicity	A transaction executes completely or not at all - no partial effects remain	In an online banking fund transfer, if money is debited from Account A but the system crashes before crediting Account B, atomicity ensures the debit is rolled back so no money disappears.
Consistency	A transaction takes the database from one valid state to another, preserving all defined rules/constraints	In an airline seat-booking system, the number of booked seats can never exceed total seats; consistency ensures this constraint always holds after every booking transaction.
Isolation	Concurrent transactions do not interfere with each other's intermediate results	In an e-commerce flash sale, if two customers try to buy the last unit of a product simultaneously, isolation ensures only one succeeds and the other sees an updated (out-of-stock) state, avoiding overselling.
Durability	Once a transaction commits, its changes persist even after a system crash or power failure	In a hospital patient-records system, once a doctor's prescription update is confirmed/committed, it must remain saved even if the server crashes a second later.

Together, these four properties are what allow application developers to treat a group of database operations as a single logical unit of work, trusting the DBMS to guarantee correctness under failures and concurrency.

Q5. Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?

Concurrency Control Problems

When multiple transactions execute concurrently without proper control, several anomalies can occur because operations from different transactions interleave:

Problem	Description	Example
Dirty Read	A transaction reads data written by another transaction that has not yet committed; if that transaction later rolls back, the first transaction has read invalid data	T1 updates a bank balance but has not committed; T2 reads this uncommitted balance; T1 then rolls back, leaving T2 with a value that never really existed
Lost Update	Two transactions read the same data and both update it based on the value they read; one update overwrites the other, and the first update is lost	T1 and T2 both read stock quantity = 10; T1 sets it to 8 (after selling 2), T2 sets it to 7 (after selling 3, based on the original 10); T1's update is lost
Unrepeatable Read	A transaction reads the same row twice and gets different values because another transaction modified and committed the data in between	T1 reads an account balance as 500; T2 updates and commits the balance to 300; T1 reads the same balance again in the same transaction and now sees 300

How Two-Phase Locking (2PL) Solves These Problems

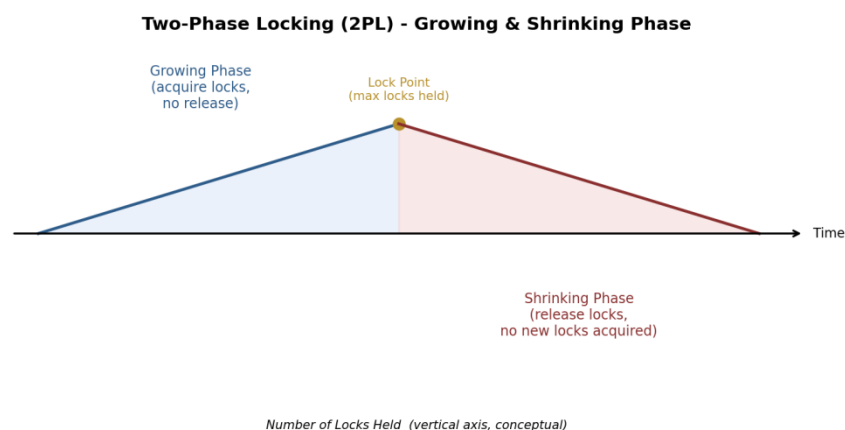


Figure 5.1: Growing and Shrinking Phases of Two-Phase Locking

Two-Phase Locking requires every transaction to acquire all the locks it needs before releasing any lock, divided into two distinct phases:

- Growing Phase: The transaction may acquire (request) locks but cannot release any lock.

- Shrinking Phase: Once the transaction releases its first lock, it can no longer acquire any new locks — it can only release locks from this point on.

By enforcing this discipline, 2PL guarantees conflict-serializable schedules, which in turn prevents the anomalies described above:

- Prevents dirty reads: A transaction must hold a shared (read) lock to read data, and a transaction cannot acquire a shared lock on data that another transaction holds an exclusive (write) lock on until that lock is released — so uncommitted writes cannot be read by others.
- Prevents lost updates: A transaction must hold an exclusive lock before writing; since only one transaction can hold an exclusive lock on an item at a time, a second transaction must wait until the first releases the lock, so its update is always based on the latest committed/locked value, not a stale read.
- Prevents unrepeatable reads: Because a transaction holds its read locks until it enters the shrinking phase (and does not acquire new locks after releasing any), no other transaction can modify data that this transaction has already read and is still relying on, so repeated reads within the same transaction return the same value.

Strict variants of 2PL (holding all locks, including read locks, until commit/abort) additionally prevent cascading rollbacks, which is why they are preferred in practical database systems (see Q10).

Q6. What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.

What is a Deadlock?

A deadlock occurs when two or more transactions are each waiting for a resource (typically a lock) held by another transaction in the same set, such that none of them can proceed. Every transaction in the cycle is blocked forever unless the DBMS intervenes, since each is waiting on a lock that will never be released.

Deadlock Detection Algorithm - Wait-For Graph

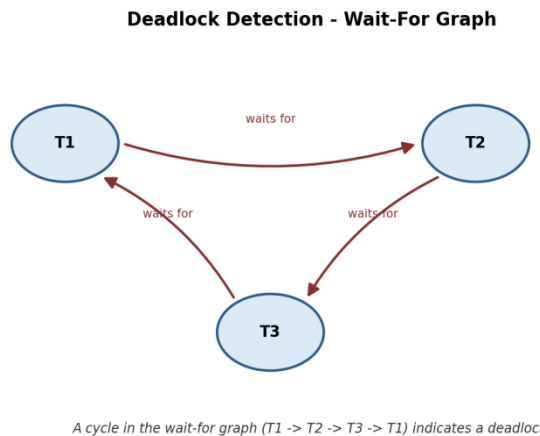


Figure 6.1: Wait-For Graph showing a Deadlock Cycle

The most common deadlock detection technique builds a wait-for graph, where each node represents a transaction, and a directed edge $T_i \rightarrow T_j$ means transaction T_i is waiting for a lock currently held by transaction T_j .

- Whenever a transaction requests a lock held by another transaction, an edge is added to the wait-for graph.
- The DBMS periodically (or on each new wait) runs a cycle-detection algorithm (e.g., depth-first search) on this graph.
- If a cycle is detected (e.g., $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1$ as shown above), a deadlock exists among the transactions in the cycle.
- The system resolves the deadlock by choosing a victim transaction (often based on age, amount of work done, or number of locks held) and rolling it back, which releases its locks and breaks the cycle.

Deadlock Prevention Strategies

Rather than detecting and breaking deadlocks after they occur, prevention strategies ensure the system never enters a deadlock state. Two widely used timestamp-based strategies are:

Strategy	Rule	Effect
Wait-Die	An older transaction (smaller timestamp) may wait for a younger one holding a lock; a younger transaction requesting a lock held by an older one is aborted (dies) instead of waiting	Only older transactions are allowed to wait, so no cycle of waiting can form (non-preemptive scheme)
Wound-Wait	An older transaction requesting a lock held by a younger one forcibly aborts (wounds)	Older transactions preempt younger ones instead of waiting

Strategy	Rule	Effect
	the younger transaction; a younger transaction simply waits for an older one	for them, again preventing any wait cycle (preemptive scheme)

Other common prevention approaches include requiring transactions to acquire all needed locks at the start (conservative 2PL, preventing partial locking), and imposing a strict global lock ordering so that all transactions request resources in the same predefined order (making a circular wait impossible).

Q7. Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?

Immediate Update vs Deferred Update Recovery

Both are logging-based recovery techniques that differ in when a transaction's changes are actually applied to the database on disk relative to when the transaction commits.

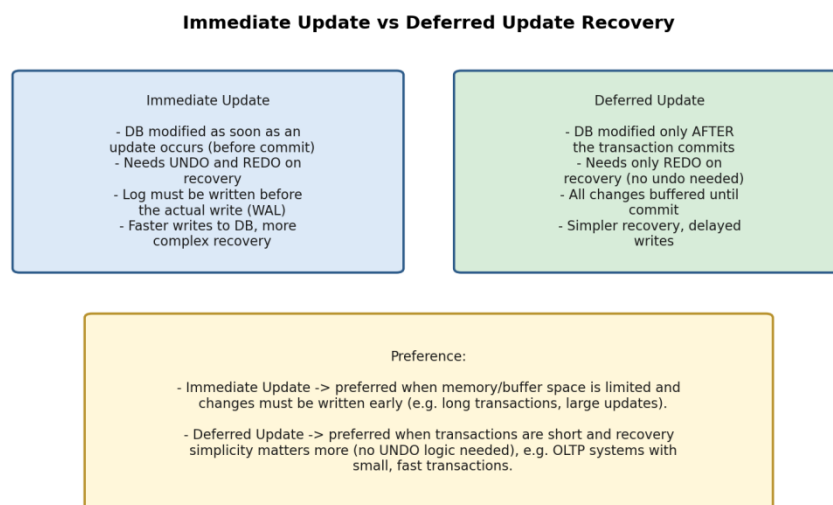


Figure 7.1: Immediate Update vs Deferred Update Recovery

Comparison Table

Aspect	Immediate Update	Deferred Update
When DB is modified	As soon as an operation executes, even before commit	Only after the transaction commits
Buffering	Changes may be written to disk during the transaction	All changes held in buffers/log until commit
Undo needed?	Yes - uncommitted changes may already be on disk	No - DB is never modified before commit
Redo needed?	Yes	Yes

Aspect	Immediate Update	Deferred Update
Logging requirement	Write-Ahead Logging (log record before actual write)	Log all changes, then apply after commit record is written
Recovery complexity	More complex (needs both undo & redo lists)	Simpler (needs only redo list)
Memory/buffer usage	Lower - writes can happen early, freeing buffers	Higher - changes held until commit

When Each Is Preferred

- Immediate Update is preferred when transactions are long-running or update large volumes of data, since holding all changes in memory until commit (as deferred update requires) would consume excessive buffer space; it also suits systems where buffer pool pressure forces early flushing of dirty pages to disk.
- Deferred Update is preferred in systems with short, fast transactions (typical OLTP workloads) where recovery simplicity is valued — since the database is never touched before commit, no UNDO logic is required at all, which simplifies the recovery manager and reduces recovery time after a crash.

In practice, most modern DBMS engines use a variant of immediate update combined with Write-Ahead Logging (WAL) and checkpointing, because it offers better performance for typical workloads while the logging protocol still guarantees correct recovery.

Q8. Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?

Shadow Paging

Shadow paging is a recovery technique that avoids the need for logging (UNDO/REDO) by maintaining two page tables during a transaction's execution: a shadow page table, which represents the last consistent (committed) state of the database, and a current page table, which is used by the running transaction and is modified as the transaction updates data.

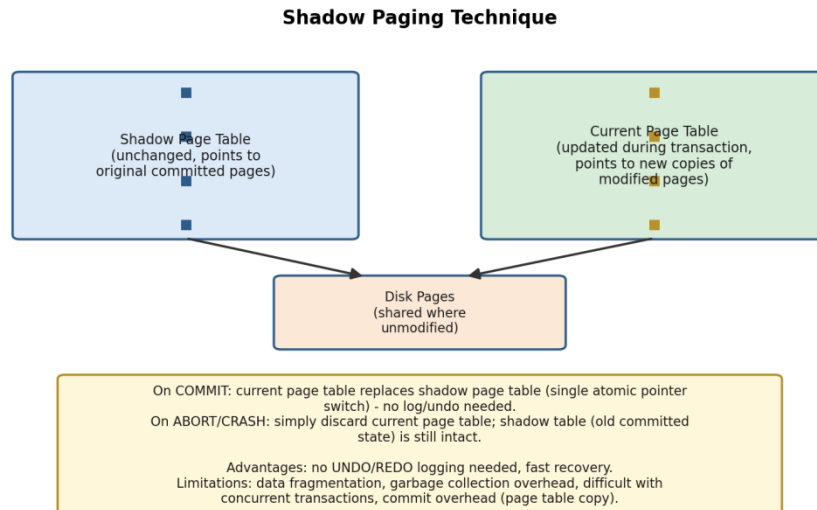


Figure 8.1: Shadow Paging - Shadow Table vs Current Table

How It Works

- At the start of a transaction, the current page table is an exact copy of the shadow page table — both point to the same physical disk pages.
- When the transaction modifies a page, a new copy of that page is created on disk, and only the current page table entry is updated to point to this new copy; the shadow page table entry is left untouched, still pointing to the original page.
- On commit, the shadow page table is atomically replaced by the current page table (a single pointer swap), making all changes permanent in one atomic step.
- On abort or a system crash before commit, the current page table is simply discarded; the shadow page table (still pointing to the original, unmodified pages) automatically restores the last consistent state — no undo operations are needed.

Advantages vs Log-Based Recovery

- No UNDO/REDO logging is required, which simplifies the recovery manager.
- Recovery after a crash is very fast — it only involves discarding the current page table, since the shadow table already represents a consistent state.
- Commit is a single atomic operation (switching the page table pointer), reducing the risk of partial commits.

Limitations vs Log-Based Recovery

- Data fragmentation: modified pages are scattered around the disk instead of being updated in place, hurting locality and sequential I/O performance.

- Garbage collection overhead: old (shadow) pages that are no longer referenced after a commit must be identified and reclaimed, adding overhead.
- Difficult to support concurrent transactions: since each transaction effectively needs its own current page table, supporting multiple concurrent transactions with shadow paging is significantly harder than with log-based schemes.
- Commit overhead: copying/updating the entire page table on commit can be costly for databases with very large numbers of pages, unlike log-based approaches which only write incremental log records.

Because of these limitations — particularly the difficulty of concurrency support — most modern DBMS engines favor log-based recovery (with WAL) over shadow paging, though shadow paging remains conceptually important and is used in some specialized/embedded systems.

Q9. What is a serializability schedule? How do you test for conflict serializability using a precedence graph?

Serializable Schedule

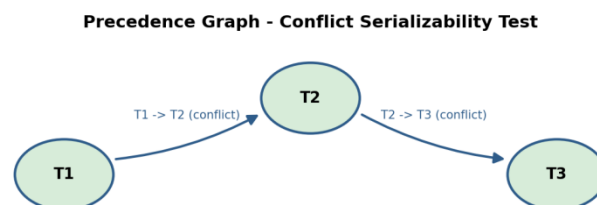
A schedule is an interleaved sequence of operations from multiple transactions executing concurrently. A schedule is said to be serializable if its effect on the database is equivalent to some serial (one-transaction-at-a-time) execution of the same transactions. Serializability is the gold-standard correctness criterion for concurrent execution, because a serial schedule is trivially correct (transactions do not interfere with each other), so any schedule equivalent to some serial order is also guaranteed correct.

There are two common notions of equivalence used to define serializability: conflict serializability (schedules are equivalent if one can be transformed into the other by swapping non-conflicting adjacent operations) and view serializability (a more general but harder-to-test notion based on equivalent reads/writes). Conflict serializability is the one most commonly tested for in practice because it has an efficient test.

Testing Conflict Serializability Using a Precedence Graph

The precedence graph (also called a serialization graph) is built as follows:

- Create one node for each transaction in the schedule.
- For every pair of conflicting operations (two operations on the same data item, from different transactions, where at least one is a write) where an operation of T_i precedes a conflicting operation of T_j in the schedule, draw a directed edge $T_i \rightarrow T_j$.
- After considering every conflicting pair, examine the resulting graph.
- If the precedence graph contains no cycles, the schedule is conflict-serializable, and a valid serial order can be obtained via a topological sort of the graph.
- If the precedence graph contains a cycle, the schedule is NOT conflict-serializable.



No cycle exists => the schedule is conflict-serializable
(equivalent to serial order T1, T2, T3).

If a cycle existed (e.g. T3 -> T1), the schedule would NOT be serializable.

Figure 9.1: Acyclic Precedence Graph Confirming Conflict Serializability

In the example above, edges $T1 \rightarrow T2$ and $T2 \rightarrow T3$ exist due to conflicting operations, and no edge closes a cycle back to an earlier transaction. Since the graph is acyclic, the schedule is conflict-serializable and is equivalent to the serial schedule $T1, T2, T3$. If, instead, an edge $T3 \rightarrow T1$ were also present, the graph would contain the cycle $T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$, and the schedule would not be serializable — no serial order could reproduce the same effect.

Q10. Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?

Two-Phase Locking Variations

All variants of Two-Phase Locking share the fundamental rule that a transaction cannot acquire any new lock after it has released a lock (growing phase followed by shrinking phase). They differ in exactly when locks are released, which affects both the strength of the correctness guarantee and the practical recoverability of the schedule.

Variant	Locking Rule	Guarantees	Drawback
Basic 2PL	Locks can be released any time after the lock point, even before the transaction commits or aborts	Guarantees conflict-serializability only	Does not prevent cascading rollbacks - if a transaction aborts after releasing a lock that another transaction already used, that transaction must also be rolled back
Strict 2PL	All exclusive (write) locks are held until the transaction commits or aborts; shared (read) locks may be released earlier	Guarantees serializability AND avoids cascading rollbacks caused by dirty writes	Slightly reduces concurrency compared to Basic 2PL since write locks are held longer
Rigorous 2PL	ALL locks (both shared and exclusive) are held until the transaction commits or aborts	Guarantees serializability, avoids cascading rollbacks, and additionally makes the commit order of transactions equal to their serialization order	Further reduces concurrency since even read locks are held for the full transaction duration

Which Variant Is Most Commonly Used?

Strict Two-Phase Locking is the variant most commonly implemented in real-world database systems. It strikes the best practical balance:

- It guarantees serializability, just like Basic 2PL.

- Unlike Basic 2PL, it avoids cascading rollbacks — because exclusive locks are held until commit/abort, no other transaction can read a value that might later need to be undone, so a single transaction's abort never forces other transactions to abort as well.
- It offers better concurrency than Rigorous 2PL, since shared (read) locks can be released earlier rather than being held for the entire transaction, allowing more transactions to read data sooner.
- It integrates naturally with standard recovery protocols (e.g., ARIES-style logging), since holding write locks until commit ensures that undo information is only ever needed for active/uncommitted transactions.

Rigorous 2PL is used in specific systems where the additional guarantee of commit-order-equals-serialization-order simplifies certain recovery or replication protocols, but it comes at the cost of reduced concurrency. Basic 2PL is mostly of theoretical interest, since production systems almost always need the cascading-rollback protection that Strict 2PL provides.